

G 弦上的魔王 (The Devil on G-String)

Modern Visual Novel Player & Server — Technical Guide

A modern, high-performance web-based player and asset compiler for the classic KiriKiri/KAG visual novel “G 弦上的魔王” (The Devil on G-String).

This project completely rewrites the original engine player, replacing it with a modular **Go standalone backend** and a responsive **React + Vite web frontend** using vanilla CSS variables, glassmorphic UI elements, and custom hooks.

Contents

🌟 Key Features (English)	1
🌟 主要特性 (Chinese)	2
📊 Runtime Emulation vs. Web-Based Renewal	3
🔧 Project Structure / 项目结构	3
🗺️ Re-Implementation Roadmap / 重新实现路线图	4
🚀 How to Build and Run / 构建与运行	4
1. Build the Binaries / 编译二进制文件	4
2. Run the Game / 运行游戏	5
⚙️ Development Server / 开发模式下运行	6
📖 Trivia & Story Notes / 游戏花絮与设定勘误	6

🌟 Key Features (English)

- **Zero-Configuration Binaries:** Single-compiled executables for Windows and Linux.
- **Automatic Asset Extraction:** The engine recursively searches the current directory (up to a depth of 3) to locate `data.xp3`. It decodes, decompresses, and extracts assets (translating `.tlg` images to standard web-ready `.webp` and `.png` on-the-fly).
- **Data Integrity Check:** Validates the presence of all **10 core game archives** (`data`, `bgimage`, `bgm`, `evimage`, `face`, `fgimage`, `image`, `others`, `sound`, `voice`) and outputs warnings if files are missing or incomplete.
- **Auto-Encoding Detection:** Supports Shift-JIS, UTF-16LE, and UTF-16BE scenario scripts. Automatically parses KAG script tags, inline furigana (`@ruby2` macros), name plates, voice clips, and page breaks (`[np]`).
- **Dynamic URL Syncing:** Automatically synchronizes your current scenario (`scen`) and dialogue index pointer (`ptr`) directly to the browser URL's query parameters (e.g. `?scen=g02&ptr=747`) for easy copying and sharing of debug bookmarks.
- **Persistent Global Settings:** Automatically saves and restores save slots, backlog history, system flags (`sf`), and volume settings (BGM, sound effects, voice) between sessions, syncing browser `localStorage` with a local `./saves.json` database.
- **Convenient Keyboard Shortcuts:**
 - **Space:** Toggles dialogue textbox visibility (to admire backgrounds and character sprites).
 - **Enter:** Advances dialogue, or makes the textbox visible again if hidden.

- **Escape**: Closes any active overlay (Settings, Save/Load slot selector, History backlog, or Flowchart).
- **I**: Toggles Immersive Subtitle (CC) Mode.
- **Q**: Quits to title menu.
- **M**: Mutes/unmutes background music.

☀ 主要特性 (Chinese)

- **免配置单文件运行**: 支持 Windows (G 弦上的魔王.exe) 和 Linux (G 弦上的魔王-server) 的一键打包与运行。
- **自动目录检测与提取**: 启动时自动在当前目录及其子目录下（深度上限为 3）递归搜索 data.xp3，自动解压并在提取时将 .tlg 图像无缝转换为可在网页加载的 .webp 和 .png 格式。
- **核心包完整性校验**: 对游戏所需的 **10 个核心存档包** (data, bgimage, bgm, evimage, face, fgimage, image, others, sound, voice) 进行检测，并在文件缺失或不完整时输出警告。
- **剧本编码自动识别**: 自动适配 Shift-JIS、UTF-16LE 及 UTF-16BE 脚本编码，支持 KAG 标记、行内注音 (@ruby2 旁注)、角色对话及换页清除符 ([np]) 的编译。
- **调试 URL 实时同步**: 自动将当前剧本文件名 (scen) 及进度指针 (ptr) 实时同步到浏览器 URL 的查询参数中（例如 ?scen=g02&ptr=747），方便复制粘贴生成调试书签。
- **存档与系统设置持久化**: 系统全局设置（音量、历史文本、章节解锁、系统标旗）与存档槽全端同步，统一由本地的 ./saves.json 进行持久化管理，即便清除浏览器缓存也不会丢失进度。
- **便捷键盘快捷键**:
 - **空格键 (Space)**: 隐藏/显示对话框（便于观赏背景 CG 及角色立绘）。
 - **回车键 (Enter)**: 推进对话，或者在对话框处于隐藏状态时重新将其显示。
 - **ESC 键 (Escape)**: 关闭当前打开的任何覆盖层模态框（系统设置、保存/读取存档、历史记录或路线图）。
 - **I 键**: 开启/关闭字幕沉浸模式。
 - **Q 键**: 返回主菜单。
 - **M 键**: 静音/恢复背景音乐。



Runtime Emulation vs. Web-Based Renewal

An engine comparison based on preservation and modern visual novel deployment models:

Feature	Low-Level (Wine/VM)	SCUMMVM Approach	Web-Based Renewal
Platform Portability	Poor (relies on OS-specific compatibility layers)	Good (cross-compiled native executables)	Excellent (zero-install, runs on mobile, desktop, tablets via browsers)
Aesthetic Upgrades	None (fixed original UI/UX constraints)	Limited (basic resolution scaling/filters)	Unlimited (CSS glassmorphism, custom fonts, blur effects)
Feature Injection	Impossible	Hard (requires patching C++ engine code)	Trivial (adding typewriters, flowcharts, instant autosaves)
Localization & Furigana	Native to original assets	Limited	Native (easy HTML <ruby> tag integration, bilingual sub-viewing)



Project Structure / 项目结构

```

.
├── cmd/
│   └── server/
│       └── main.go      # Go entry point (API server, HTTP routes, assets)
├── lib/
│   └── extraction/      # Go extraction package / 资源提取处理包
│       ├── extractor.go # extraction coordinator and path walker
│       ├── filemap.go   # asset file-mapping indexer
│       ├── pos.go       # sprite coordinate parser
│       ├── scenario.go  # scenario parser and compiler
│       └── xp3.go        # KiriKiri XP3 archive decompressor
├── web-app/             # React frontend / React 前端
│   ├── src/
│   │   ├── hooks/
│   │   │   ├── useKagRunner.js # KAG game loop state machine
│   │   │   └── useGameAudio.js # HTML5 audio manager
│   │   └── App.jsx             # main layout & overlays router
│   └── vite.config.js          # Vite build configuration
├── build.sh               # Compilation script
├── G弦上的魔王-server     # Compiled Linux binary
├── G弦上的魔王.exe        # Compiled Windows binary
└── saves.json             # Local state database

```



Re-Implementation Roadmap / 重新实现路线图

A four-stage pipeline for Visual Novel preservation and modern web deployment:

Stage 1: Asset Pipeline (Extractor)

Decrypt and unpack proprietary archives (XP3/BSA/DAT), convert raw images to web-optimized formats (PNG/WebP), and standardize audio.



Stage 2: Scenario Compiler (Transpiler)

Parse scenario text files (CP932/Shift-JIS), map engine commands (sprites, transitions, layers) to a unified JSON instruction format.



Stage 3: Unified React VM Player

Render character layers using viewport layout matrices, manage loopable audio channels, and display modern UI overlays.



Stage 4: Local Server

Run the Go-based standalone server to serve resources, handle file saving/loading, and persist states to saves.json.



How to Build and Run / 构建与运行

1. Build the Binaries / 编译二进制文件

Ensure you have Go and Node.js (with pnpm or npm) installed. / 请确保已安装 Go、Node.js 及 pnpm 包管理器。

To build the React application and compile the server binaries:

```
# 1. Build the production React frontend / 构建前端资源
cd web-app
pnpm install
pnpm build
cd ..

# 2. Compile Go binaries / 交叉编译后端二进制文件
./build.sh
```

This generates G 弦上的魔王-server (for Linux) and G 弦上的魔王.exe (for Windows) in the root folder.

2. Run the Game / 运行游戏

Double-click G 弦上的魔王.exe (Windows) or execute ./G 弦上的魔王-server (Linux). / 双击 G 弦上的魔王.exe (Windows) 或执行 ./G 弦上的魔王-server (Linux)。

The server will automatically:

1. Search for the game directory recursively. / 自动递归搜索游戏目录。
2. Inspect archives for completeness. / 自动校验游戏文件完整性。
3. Perform extraction and compile scenarios if they aren't already extracted. / 如果未提取，则自动提取资源并编译剧本。
4. Bind to port 8080. / 绑定端口 8080。
5. **Automatically launch your default web browser** to <http://localhost:8080> to start playing! / **自动调用系统浏览器**打开 <http://localhost:8080> 开始游玩!

⚙️ Development Server / 开发模式下运行

To run a hot-reloaded development environment concurrently with a single command: / 仅需一条命令即可同时启动支持热更新的前后端本地开发服务：

```
# Start backend server and automatically spawn Vite dev server in the background
./G弦上的魔王-server -dev
```

Alternatively, you can start them manually in separate terminals: / 或者，你也可以在不同的终端中手动分别启动它们：

1. Start the Go backend server (handles data APIs and assets serving): / 启动 Go 后端服务：

```
./G弦上的魔王-server -port 8080
```

2. Start the Vite frontend development server: / 启动前端开发服务：

```
cd web-app
pnpm dev
```

3. Vite will proxy all static visual novel assets (/bgimage/, /bgm/, etc.) and API requests back to the Go server on port 8080. / Vite 会自动将所有的静态资源与 API 请求代理至本地的 Go 服务端口（8080）。

📖 Trivia & Story Notes / 游戏花絮与设定勘误

English Trivia

- **Air on the G String (G 弦上的咏叹调)**: A real classical violin arrangement by August Wilhelmj (1871) of the second movement of Johann Sebastian Bach's **Orchestral Suite No. 3 in D major** (BWV 1068). Wilhelmj transposed the piece to C major and adapted it so that a violinist can play the entire melody using **only the lowest string (the G string)**.
- **Historical Fictionalization (Scenario g10)**: In scenario g10 (pointers 943–954), Kyousuke claims that “Air on the G String” was directly played as a requiem at a major US terrorist attack (9/11). This detail is **fictionalized** for the story. Historically, Yo-Yo Ma performed Bach's **Cello Suite No. 1** at the Ground Zero anniversary service, and Samuel Barber's **Adagio for Strings** remains the piece of music most famously associated with 9/11 commemorations.

中文花絮与勘误

- **G 弦上的咏叹调 (Air on the G String)**: 历史上真实存在的小提琴改编曲。由德国小提琴家奥古斯特·威尔赫米 (August Wilhelmj) 于 1871 年将巴赫的《D 大调第三管弦乐组曲》(BWV 1068) 的第二乐章进行改编，通过降调（至 C 大调）并将旋律下移八度，使得整首乐曲可以**仅在小提琴最粗的 G 弦上演奏完毕**。
- **剧情历史勘误 (剧本 g10)**: 在剧本 g10（指针 943–954）中，京介提到“G 弦上的咏叹调曾在美国大型恐怖袭击（即 9/11 事件）中被直接用作镇魂曲演奏”。该说法为游戏故事的**艺术虚构**。历史上，9/11 双子塔遗址（Ground Zero）首周年纪念仪式上演奏的最著名的巴赫乐曲是马友友演奏的《第一号无伴奏大提琴组曲·前奏曲》，而与 9/11 纪念活动关联最深、最著名的“非官方镇魂曲”则是塞缪尔·巴伯的《弦乐慢板》(Adagio for Strings)。